DATA MANAGEMENT & KNOWLEDGE GRAPHS

Organiser la donnée

Des documents dispersés à un graphe de connaissances **sourcé, visualisé et reconstructible**.

SOURCES · CURATION · DOMAINE · KNOWLEDGE · NEO4J AURA



Photo Luke Chesser / Unsplash

POINT DE DÉPART

Avoir les documents suffit-il ?

retrouver

relier

prouver

LA PROMESSE

Lire une architecture data, puis construire son graphe.

01



Sources

Identifier l'origine, la version et l'autorité.

02



Curation

Nettoyer, normaliser et conserver la provenance.

03



Domaine

Nommer les concepts, identités et règles métier.

04



Graphe

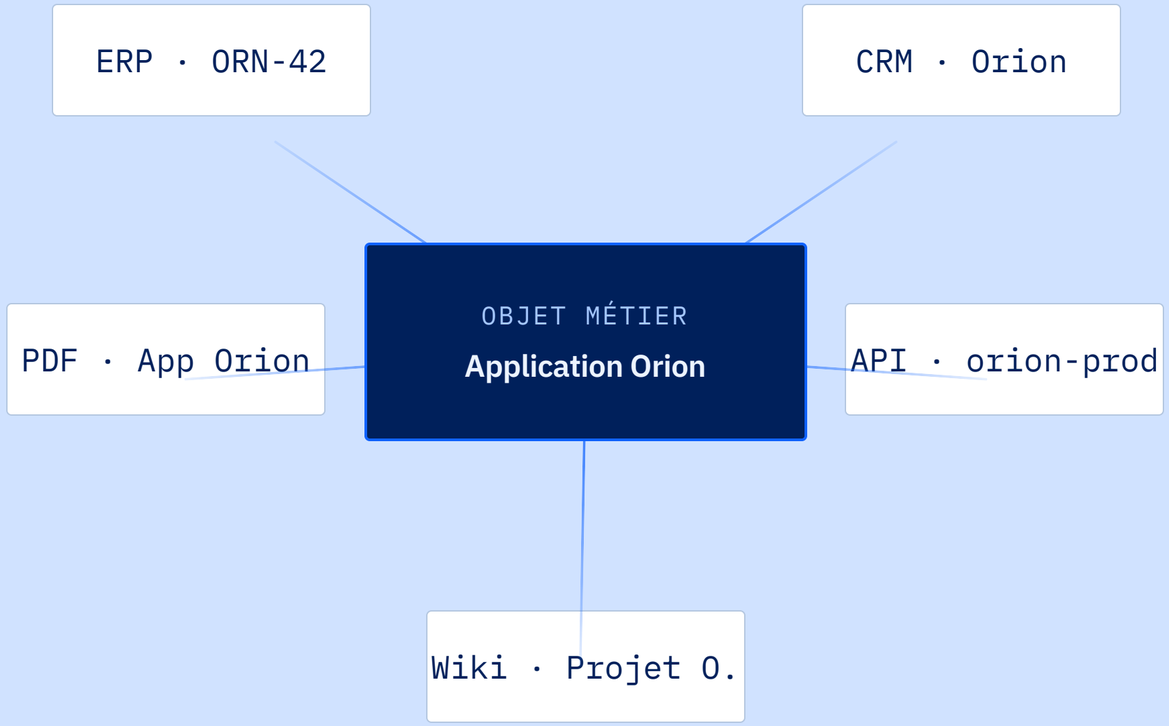
Transformer les faits en relations explicites.

05



Validation

Rejouer le script et vérifier les chemins.



CONTEXTE D'ORGANISATION

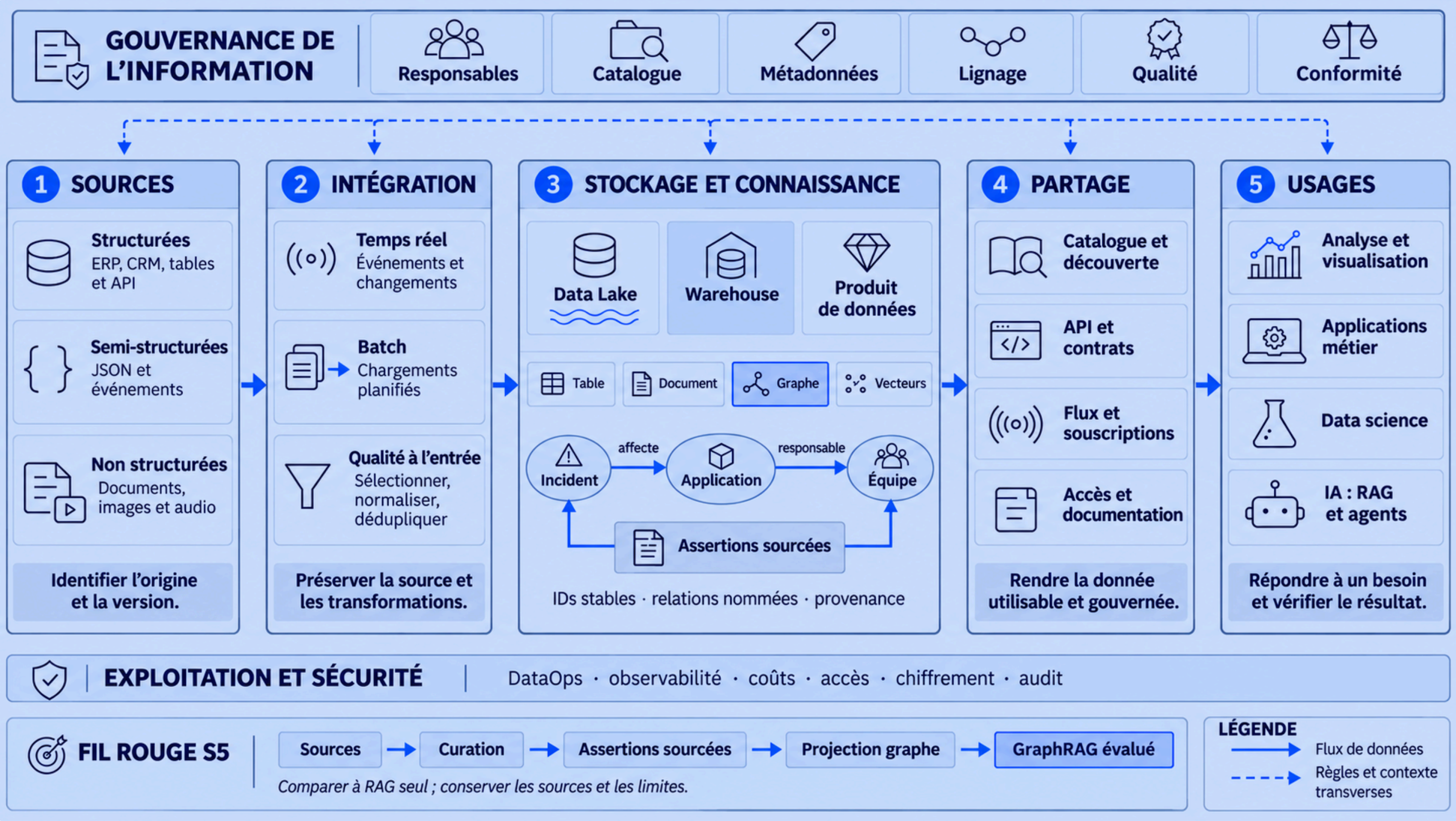
Beaucoup de données. Pas encore une vérité partagée.

Les systèmes décrivent parfois le même objet avec des identifiants, des formats et des propriétaires différents.

Notre travail commence par rendre ces différences **explicites et traçables**.

C'est normal de ne pas tout comprendre maintenant.

À la fin de la séance, vous saurez raconter le trajet de gauche à droite — jusqu'au graphe construit.



MODE D'EMPLOI

Une architecture se lit dans plusieurs directions.



Horizontal

De la source brute jusqu'à un usage utile.



En haut

Gouvernance, metadata, qualité et lineage.

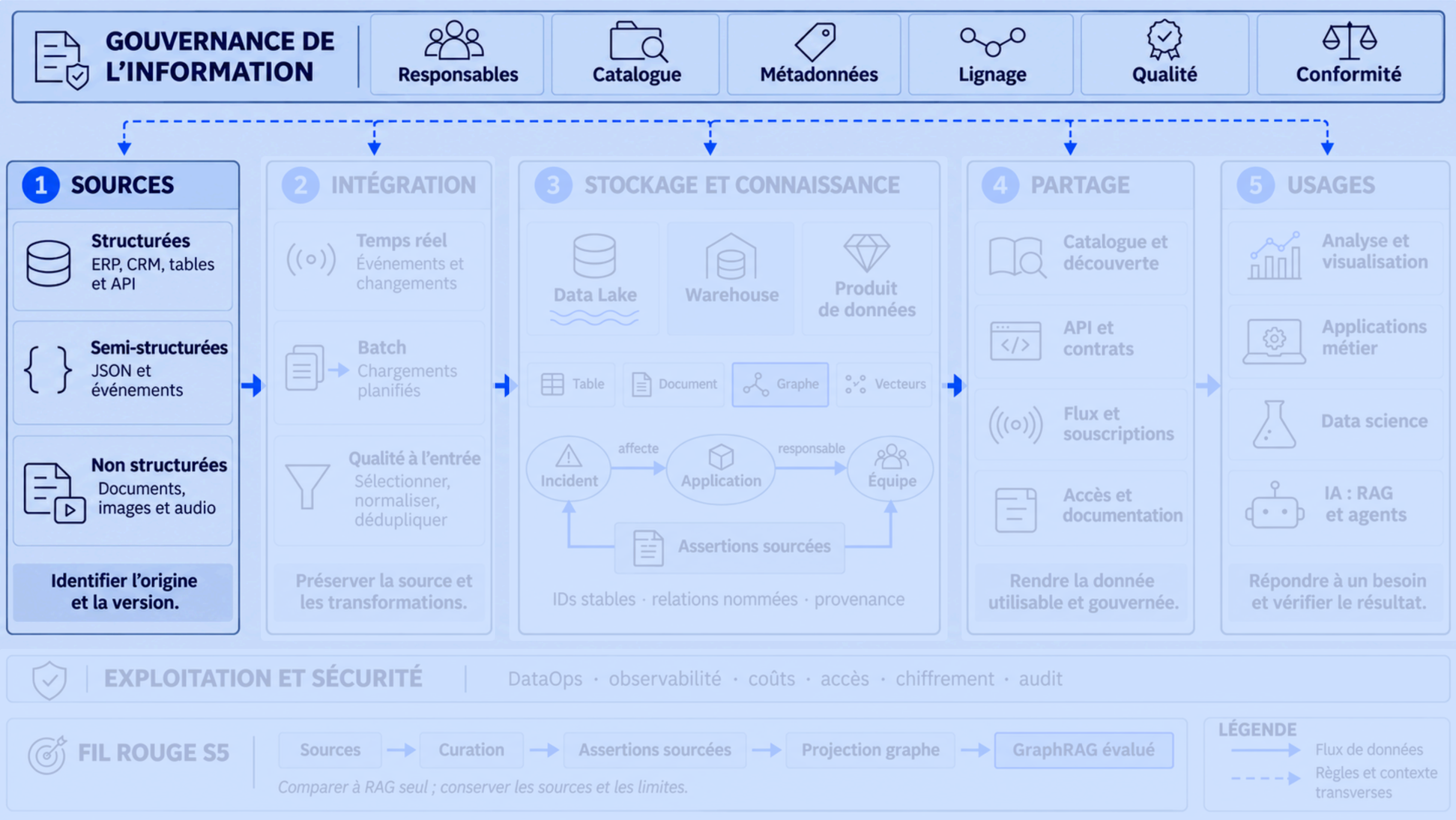


En bas

Sécurité, exploitation et responsabilités transverses.

Toute donnée commence quelque part.

Avant de transformer : identifier l'autorité, la version et les conditions d'usage.



PAS LE MÊME MATÉRIAU

Toutes les données ne se ressemblent pas.

Structuré

Schéma explicite, colonnes attendues, contraintes possibles.

```
id | application | owner
42 | Orion | Payments
```

Semi-structuré

Structure présente, mais flexible ou imbriquée.

```
{
  "app": "Orion",
  "tags": ["critical"]
}
```

Non structuré

Le sens est porté par le texte, l'image, l'audio ou la mise en page.

```
« L'équipe Payments est
responsable de l'application
Orion. »
```

MÊME RÉALITÉ, PLUSIEURS VUES

On choisit une représentation selon la question.



Table

Pour filtrer, agréger et appliquer des contraintes structurées.

```
SELECT owner,
COUNT(*)
FROM applications
GROUP BY owner;
```



Document

Pour lire un contenu riche dans son contexte.

```
Titre
Résumé
Décisions
Exceptions
Justifications
```



Graphe

Pour rendre les relations explicites et parcourir des chemins.

```
(Incident)
  —AFFECTS→
(Application)
  —OWNED_BY→ (Team)
```

TRANSFORMER SANS PERDRE L'ORIGINE

Curater, c'est construire la confiance.

ENTRÉE

Données brutes

Doublons, alias, champs absents, formats hétérogènes.

→

sélectionner

normaliser

dédupliquer

enrichir

valider

→

SORTIE

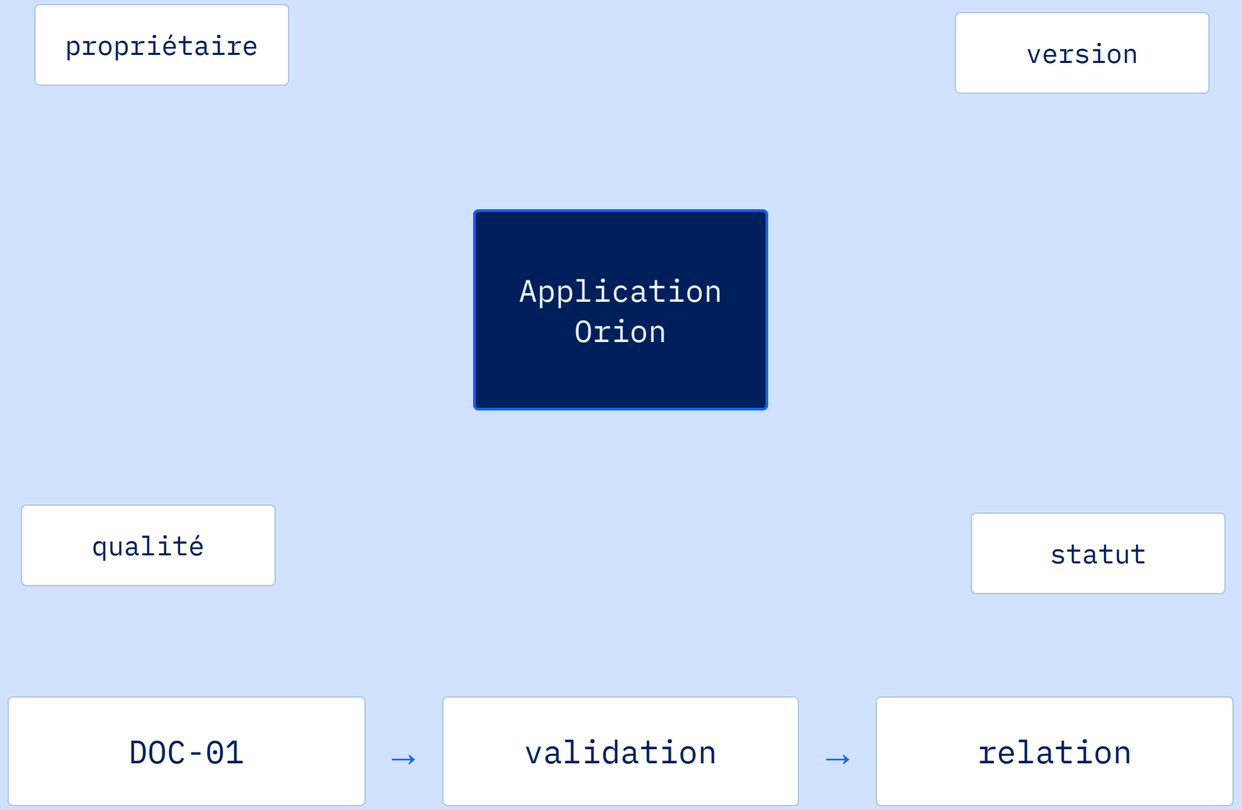
Donnée qualifiée

Identité, qualité, version et provenance explicites.

LA DONNÉE SUR LA DONNÉE

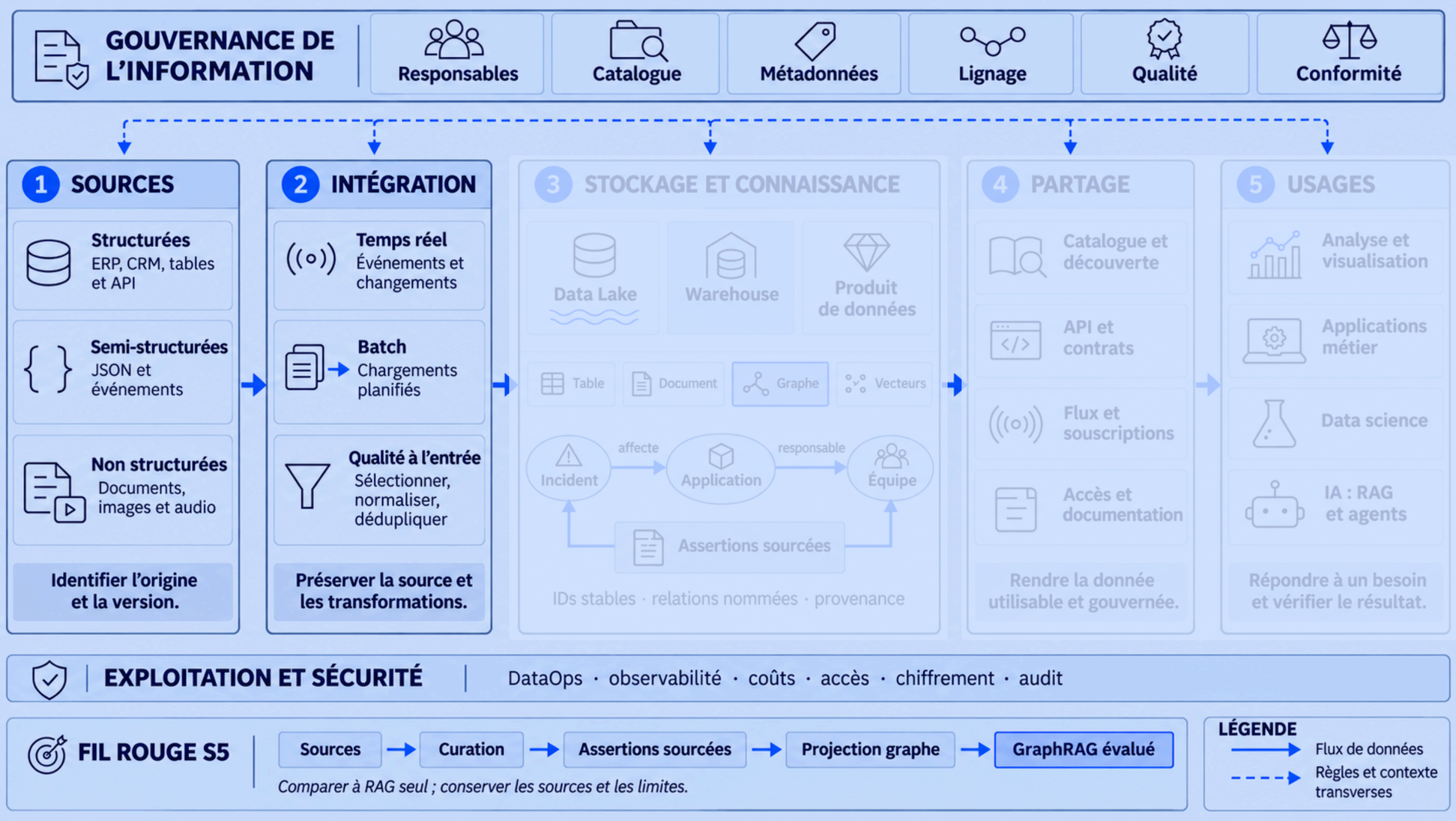
Savoir ce que c'est, qui en répond, et d'où cela vient.

Une transformation utile doit rester explicable : on peut remonter du résultat à la source et à la règle appliquée.



La confiance se construit avant le stockage.

Sources, intégration, curation et gouvernance forment une continuité.



CHANGEMENT DE REGARD

Le domaine n'est pas la structure des documents.

LE FICHIER

Organisation éditoriale

DOC-042 version 7 · PDF

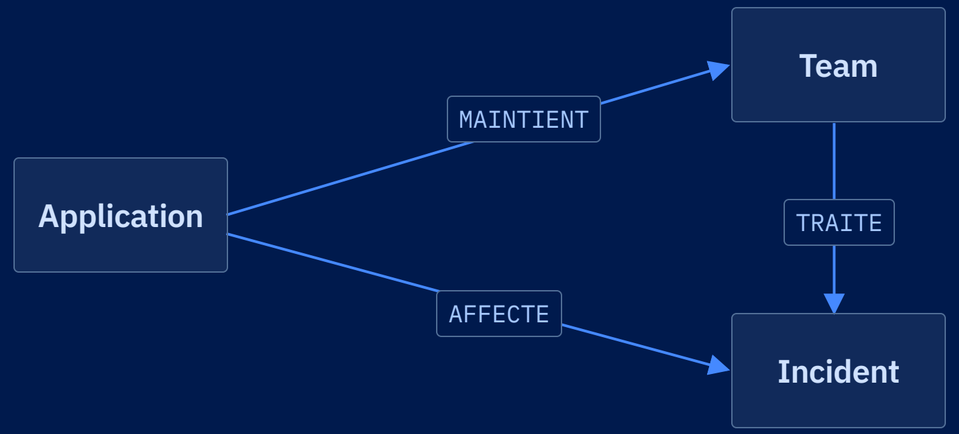
1. Introduction
2. Périmètre
3. Responsabilités
4. Exceptions

Une section n'est pas automatiquement une entité métier.

≠

LE DOMAINE

Concepts et relations



DU DOMAINE À SA PROJECTION

Les mots du métier restent indépendants de la technologie.

Neo4j / Aura
reste à l'extérieur



LIRE, QUALIFIER, STRUCTURER

Un document contient des affirmations.

« Depuis janvier 2026, **l'équipe Payments est responsable de l'application Orion**. »

DOC-01 · version 3 · § Responsabilités

→

ASSERTION VALIDÉE

Orion — OWNED_BY → Payments

sourceId = DOC-01
evidence = passage exact

UN OBJET, PLUSIEURS NOMS

Les alias convergent vers un identifiant stable.

« Orion »

« App Orion »

« ORN-42 »

→

APPLICATION

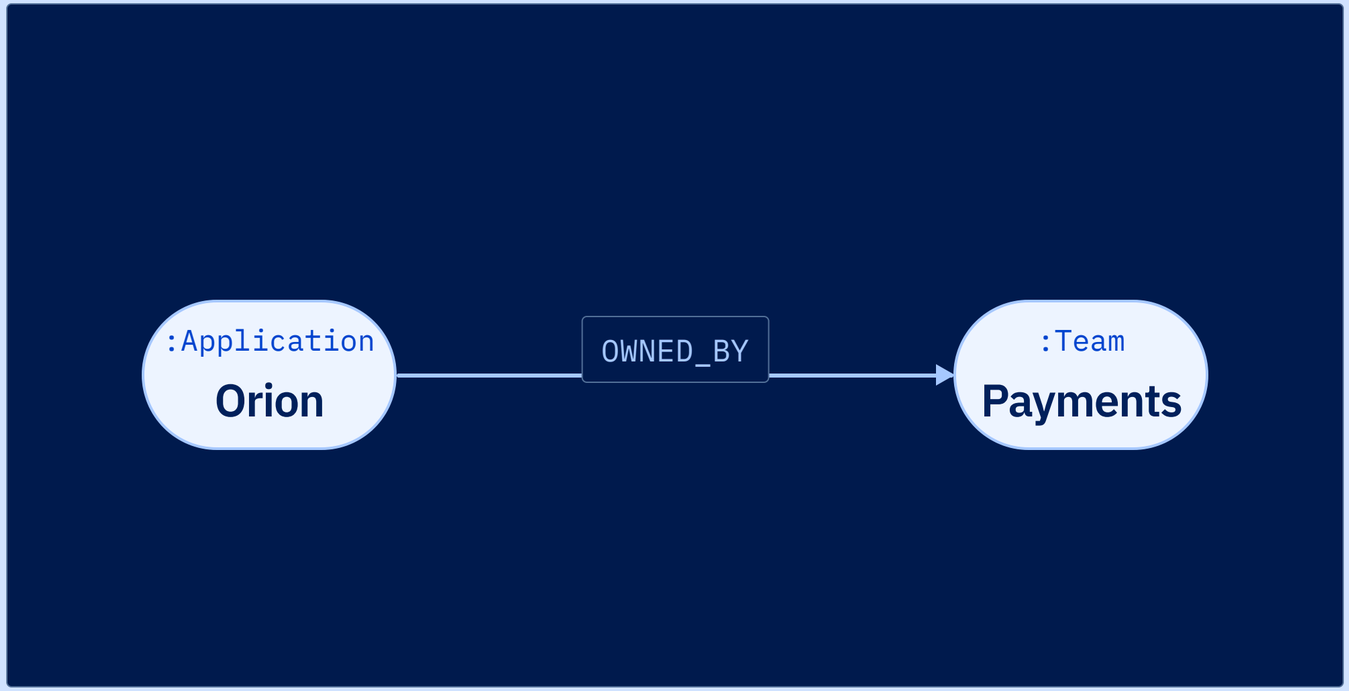
Orion

id = app-orion

Sans résolution d'identité : trois faux nœuds · trois vérités divergentes

PEU DE PRIMITIVES, BEAUCOUP DE SENS

Nœuds, relations, propriétés et direction.



NODE · une chose avec une identité.

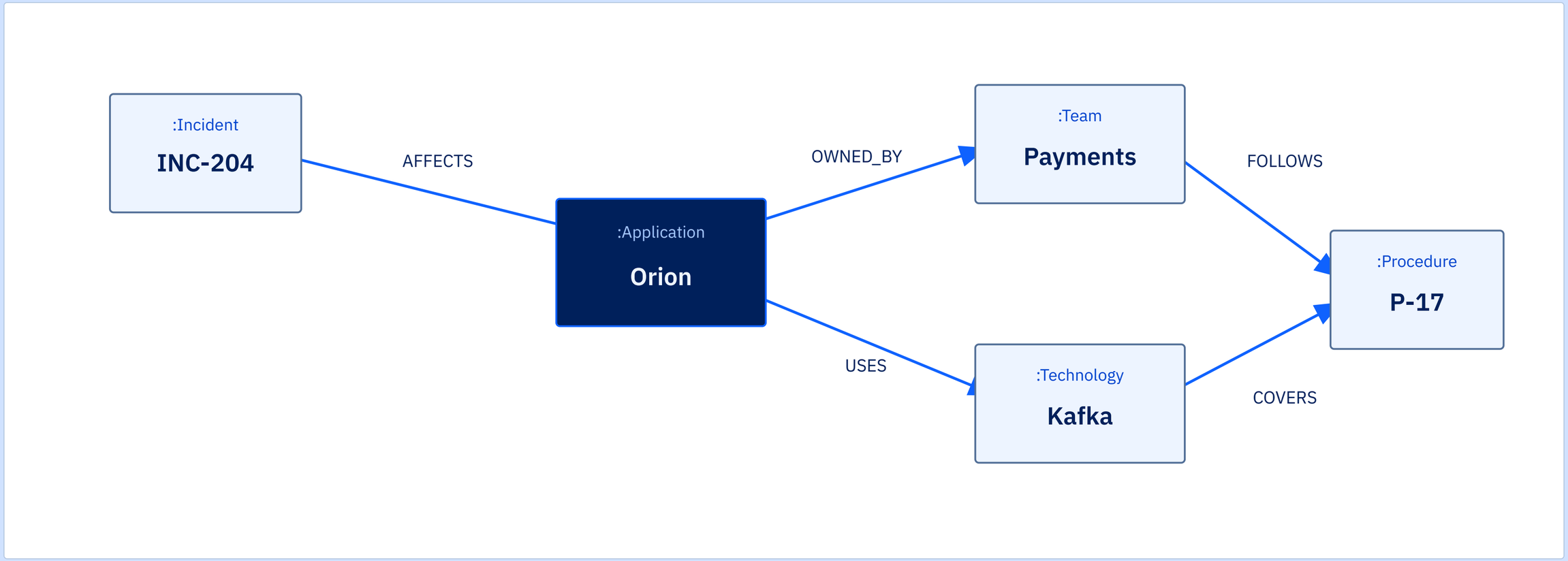
LABEL · son type dans la projection.

RELATION · un verbe métier orienté.

PROPERTY · une valeur portée par un nœud ou une relation.

UN MODÈLE VOLONTAIREMENT PETIT

Incidents, applications, équipes et technologies.



LA QUESTION JUSTIFIE LE GRAPHE

Une bonne question traverse les relations.

1 saut

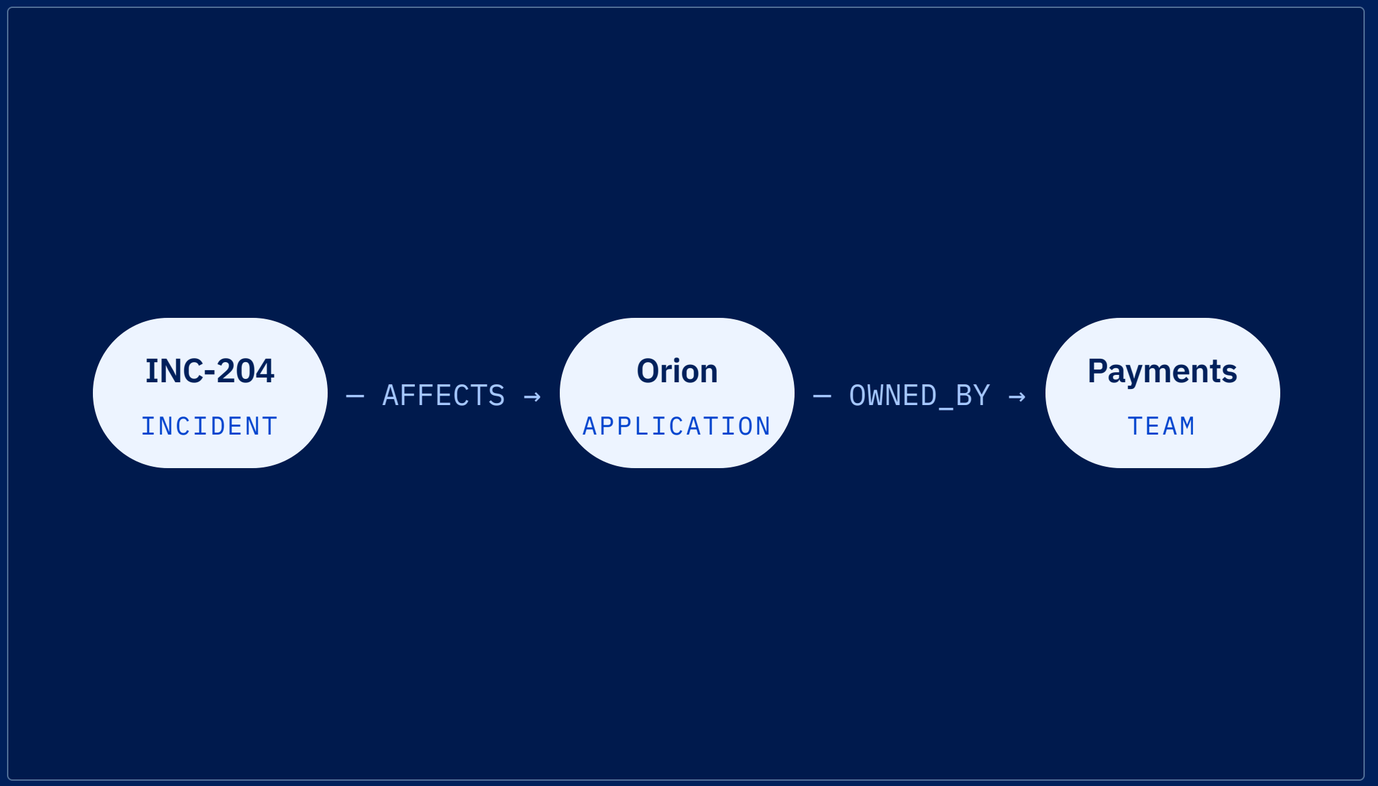
Quelle application est affectée par l'incident INC-204 ?

2 sauts

Quelle équipe possède l'application affectée par INC-204 ?

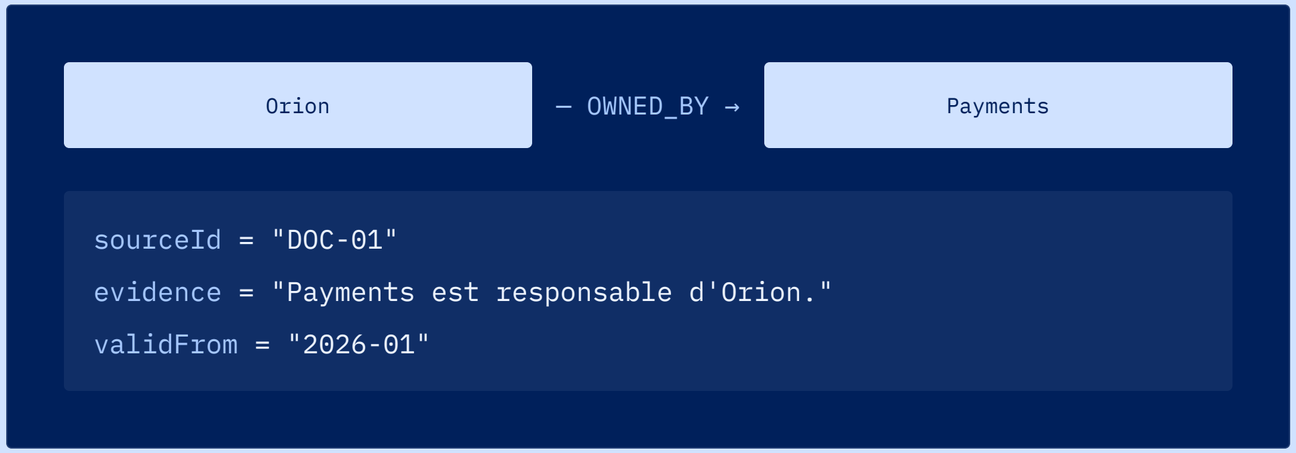
Contrôle

Quelles sources justifient chaque relation du chemin ?



UNE RELATION N'EST PAS DÉCORATIVE

Une relation sans preuve reste fragile.



FAIT

Présent explicitement dans une source identifiée.

INTERPRÉTATION

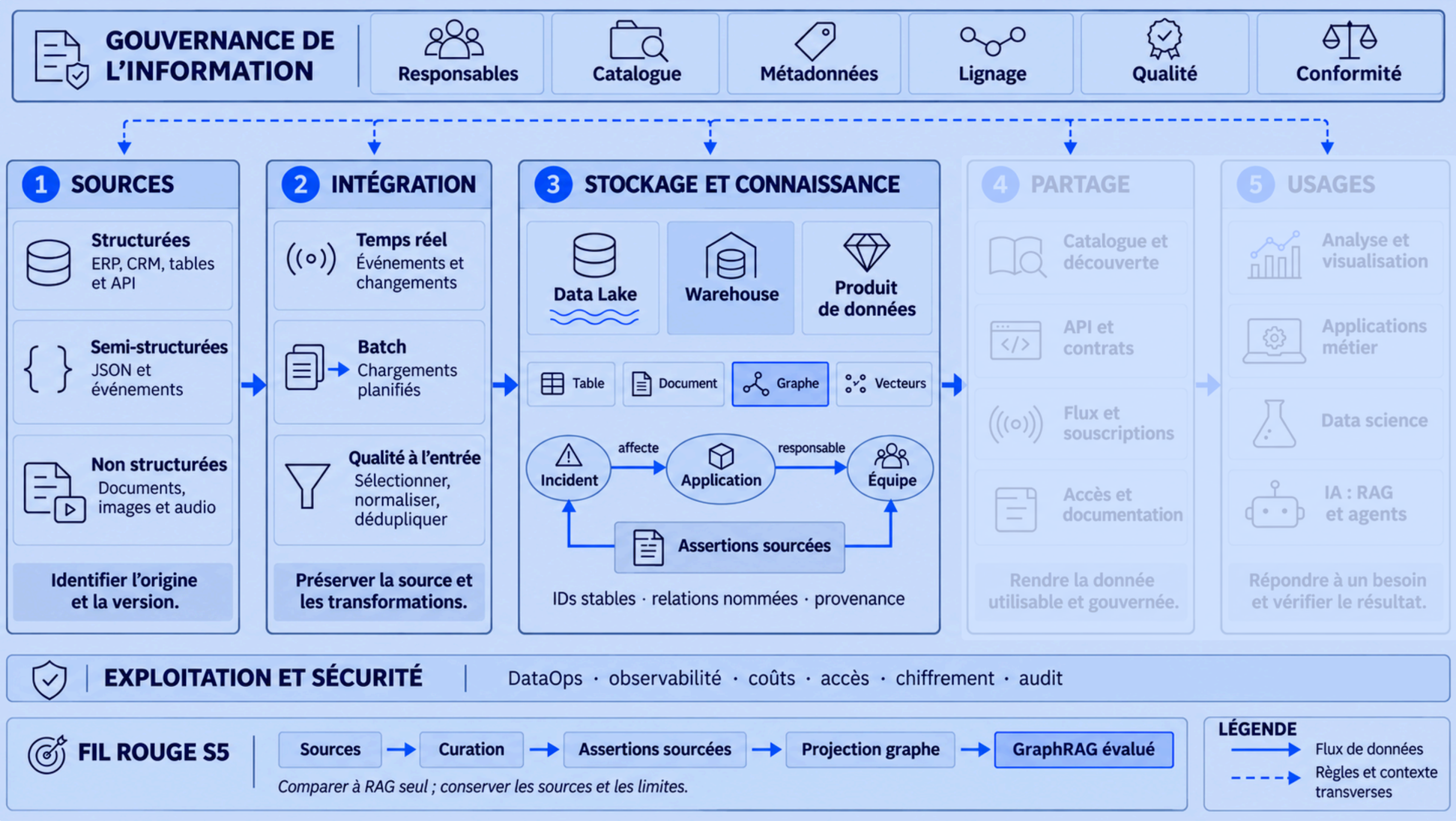
Choix du groupe, documenté comme tel.

INCONNU

L'information manque : on ne l'invente pas.

Les assertions validées deviennent une projection graphe.

Le sens vient du domaine ; Neo4j stocke une projection reconstruite depuis des artefacts versionnés.



AU-DELÀ DE LA BASE

Une base de données n'est pas encore un produit.



POUR QUI ?

Des utilisateurs

Des besoins et des questions identifiés.

PAR QUI ?

Un responsable

Quelqu'un répond de la qualité et de l'évolution.

COMMENT ?

Un contrat

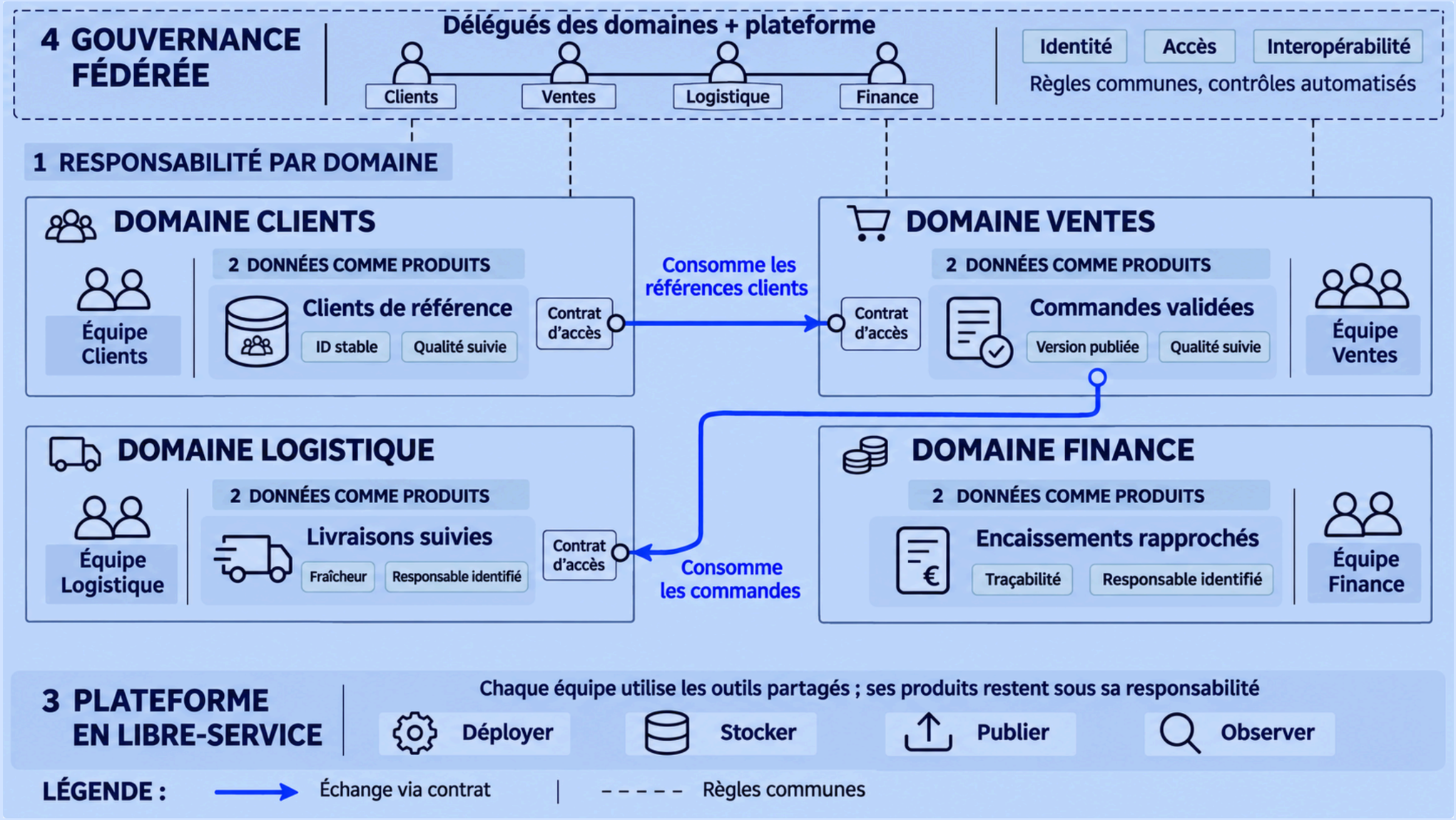
Schéma, accès, documentation et règles d'usage.

AVEC QUELLE CONFIANCE ?

Une qualité visible

Fraîcheur, complétude, provenance et limites.

Distribuer la responsabilité, garder des règles communes.



UN VRAI MOTEUR, SANS SERVEUR LOCAL

Pourquoi Aura pour le TP ?

Neo4j Aura

Construire, requêter et visualiser le graphe directement dans le navigateur.

Query | Cypher | Graph view | Export PNG

INSTALLATION

Aucun serveur local

APPRENTISSAGE

Neo4j et Cypher réels

PRÉREQUIS

1 instance / groupe, prête avant S5

UN SCRIPT, TROIS RESPONSABILITÉS

Le graphe doit être rejouable.

01 · CONSTRAINT

```
CREATE CONSTRAINT
application_id
IF NOT EXISTS
FOR (a:Application)
REQUIRE a.id IS UNIQUE;
```

Protéger l'identité et empêcher les doublons.

02 · MERGE

```
MERGE (a:Application {
  id: "app-orion"
})
SET a.name = "Orion";
```

Créer ou retrouver sans dupliquer au second passage.

03 · MATCH

```
MATCH path =
  (:Incident)-[:AFFECTS]->
  (:Application)-[:OWNED_BY]->
  (:Team)
RETURN path;
```

Contrôler que le chemin attendu existe réellement.

MISSION

Du domaine S4 au graphe, puis à GraphRAG.

VOTRE DÉFI

Construire un graphe sourcé et l'utiliser dans GraphRAG.

À partir du domaine métier construit en S4 et de documents publics ou synthétiques : définir la projection, écrire le script Cypher, valider les chemins, puis comparer RAG seul et RAG enrichi par le graphe.

- `graph_model.md`
- `graph.cypher`
- capture PNG
- 2 chemins validés
- comparaison RAG / GraphRAG



UN CHEMIN CONTRÔLÉ

80 minutes pour construire, connecter et comparer.

05'	10'	15'	15'	10'	15'	10'
Connexion	**Projection**	**Nœuds**	**Relations**	**Validation**	**GraphRAG**	**Comparaison**
Ouvrir Aura et exécuter la requête de test.	Projeter le domaine S4 en labels et relations.	Créer contraintes et entités avec MERGE.	Ajouter sourceId et evidence.	Visualiser et vérifier les chemins.	Résoudre, parcourir 1–2 sauts et enrichir le contexte.	Comparer RAG seul, GraphRAG et fallback.

IDs stables | provenance | chemin à 2 sauts | script rejouable

Le graphe est construit, validé et consommé avec mesure.

Vous savez reconstruire la projection, borner les parcours et reconnaître les questions pour lesquelles GraphRAG n'apporte rien.

